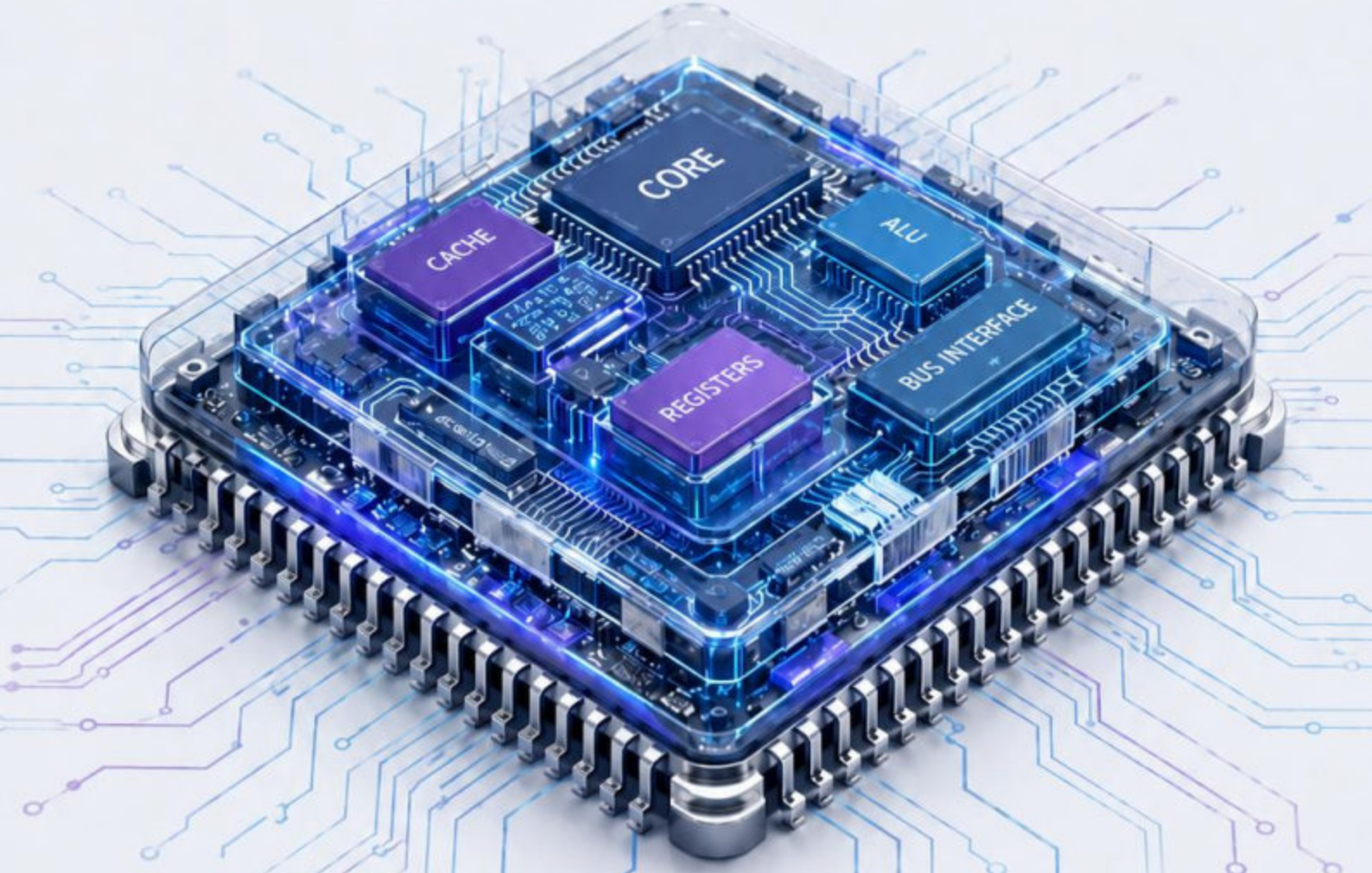
فريق نون التطوعي - السنة الرابعة

المعالجات الصغرية و نظمها

الملحق الخامس :

المعالجات الصغرية و بنيتها

قسم د.مازن محايري



كتابة : سالي السبسبي
تدقيق : عمر زين
تفريغ : ليلاس إدلبي-حنين أحمد

قسم هندسة الالكترونيات والاتصالات
2025 - 2026

النموذج البرمجي لمعالجات _ (CISC)

وحدة المعالجة المركزية CPU :

تعريفه: هو أداة متكاملة وظيفتها معالجة البيانات و تنفيذ الأوامر ، وتتكون من :

.وحدة للحساب والمنطق (ALU) .

وحدة التحكم(CU)

مسجلات (Registers)

((جميعها داخل شريحة واحدة))

سؤال:

ما الفرق بين المعالج الصغري والمتحكم الصغري ؟

الحل:

المعالج:microprocessor

1- يعتمد على مكونات خارجية كوحدات الإدخال والإخراج.

2- ينفذ ويعالج فقط.

3- أداؤه عالٍ.

.4- استهلاك عالٍ للطاقة.

المتحكم:microcontroller

1- يحوي معالج ومنافذ إدخال وإخراج وذاكرة على نفس الشريحة (نظام متكامل).

2-. أداء متوسط.

3- استهلاك منخفض للطاقة.

مثال عن المتحكمات الصغرية: الغسالات - أنظمة التحكم في السيارات.

The Intel Family

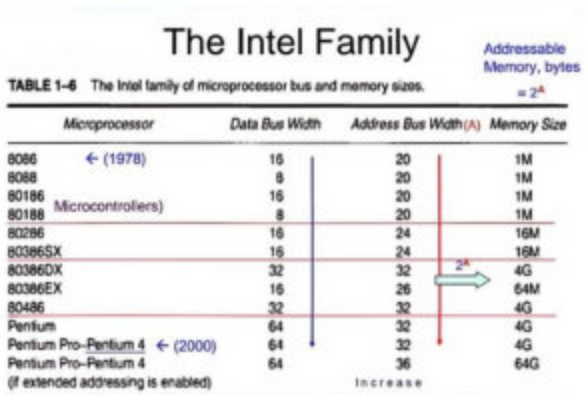

The Intel Family

TABLE 1-6 The Intel family of microprocessor bus and memory sizes.

Microprocessor	Data Bus Width	Address Bus Width (A)	Memory Size
8086 ← (1978)	16	20	1M
8088	8	20	1M
80186 Microcontrollers)	16	20	1M
80188	8	20	1M
80286	16	24	16M
80386SX	16	24	16M
80386DX	32	32	4G
80386EX	16	26	64M
80486	32	32	4G
Pentium	64	32	4G
Pentium Pro–Pentium 4 ← (2000)	64	32	4G
Pentium Pro–Pentium 4 (If extended addressing is enabled)	64	36	64G

Addressable Memory, bytes = 2^A

2^A

Increase

النموذج البرمجي Programming Model

تعريفه: هو الصورة الذهنية للمعالج من منظور البرمجة أي هو وصف لكيفية تعامل المبرمج مع المعالج.

سنبدأ بدراسة النموذج البرمجي لمعالج (Intel 8086)

أولاً المسجلات Registers :

و تنقسم إلى :

- مسجلات أغراض عامة
- مسجلات أغراض خاصة



✨مسجلات الأغراض العامة:

✨مسجل المراكم العام AX (Accumulator Registers: 16 Bit):

مسجل هام يستخدم في العمليات الحسابية وهو مسجل أغراض عام له بعض الوظائف الخاصة.

AH (جزء علوي) 8bit	AL (جزء سفلي) 8bit

✨مسجلBX (Base Registers : 16 Bit):

يستخدم لتخزين عناوين الذاكرة.

BH (جزء علوي) 8bit	BL (جزء سفلي) 8bit

✨مسجل CX (Count Registers : 16 Bit):

يستخدم كعداد تلقائي في العمليات التكرارية و يستخدم في الحلقات التكرارية .

CH (جزء علوي) 8bit	CL (جزء سفلي) 8bit



✨ مسجل DX :(Data Registers : 16 Bit):

له وظيفتين أساسيتين :

- يحتفظ بالباقي الذي لم يتسع بمسجل المراكم (بالقسمة) و يخزن الجزء الأعلى (بعملية الضرب).
- يتم فيه تخزين القيم المقروءة من وحدات الإدخال والإخراج .

مثال : قراءة قيمة من لوحة المفاتيح يتم تخزينها في سجل الداتا .

DH (جزء علوي) 8bit	DL (جزء سفلي) 8bit

DI(destination index) : يتم فيه تخزين عنوان مقطع الذاكرة في حال كانت التعليمة String .

✨مسجلات الأغراض الخاصة : *(16 Bit)* ✨

IP(instruction pointer): يشير إلى العنوان في الذاكرة حيث توجد التعليمة التالية التي يجب تنفيذها.

بعد تنفيذ كل تعليمة، يتم تحديث قيمة الـ*IP* تلقائيًا ليشير إلى التعليمة التالية

SP(stack pointer) :يشير إلى أعلى موقع في المكدس *(*Stack*)* داخل الذاكرة .المكدس يُستخدم لتخزين بيانات مؤقتة مثل:

- العناوين المرجعية عند استدعاء الدوال.*(*Return addresses*)*
- المتغيرات المحلية.
- حفظ محتويات بعض السجلات أثناء المقاطعات أو الاستدعاءات.

flags

ملاحظة :كلما زادت مسجلات الإزاحة (Offset) كلما كانت كتابة البرنامج أسهل على المبرمج.

ملاحظة : في المعالجات الحديثة تم توسيع مسجلات الأغراض العامة لتصل 32 بت .لأننا نتعامل مع داتا أكبر

مثال:EAX (الموسع E = Exteuted) : هو نفسه مسجل AX بالإضافة إلى 16 بت إضافية

16 bit	AH 8bit	AL 8bit

ملاحظة : من معالجات Intel 8086 وصولاً إلى معالجات البنتيوم P4 (32bit)
من PU إلى معالجات Core i (64 bit .)

ثانياً : مقاطع الذاكرة

تتألف الذاكرة من أربع مقاطع مسجلات (segments) :

Code
Data
Stack
Extra

1. CS , IP
2. DS , BX , SI , DI
3. SS , SP , BP
4. ES , DI

BP(Base pointer) : مسجل يستخدم للوصول إلى المتغيرات داخل المكدس.

SI(Sourse index) : يستخدم كمؤشر إلى مصدر البيانات (مثل عمليات النسخ).

DI(destination index) : يستخدم كمؤشر إلى وجهة البيانات .

شرح المقاطع :

- Code Segment : مكان في الذاكرة يتم فيه تخزين الأكواد و الأوامر في الذاكرة RAM.
- Data Segment : يتم فيه تخزين المعلومات ويكون مصدراً لها

ملاحظة : يفضل فصل الCODE عن الDATA ، لنستطيع تغيير الداتا دون تغيير الكود.

- Stack Segment : وظيفته التخزين المؤقت السريع للداتا و يتم فيه تخزين عناوين عودة (في أثناء استخدام الدوال) وهو مقطع هام جداً لتشغيل البرامج .
- Extra Segment : هو داتا تستخدم فقط أثناء كتابة تعليمات String (يستقبل البيانات القادمة من وحدات الإدخال و تخزن في هذا المقطع).

ملاحظة: لزيادة عدد الداتا المخزنة تم إضافة مقطعين إضافيين في المعالجات الحديثة (GS , FS)



العنونة addressing:

هي طريقة يحدد بها المعالج مكان البيانات في الدارة.

سؤال : كيف تتم عملية العنونة ؟

الجواب : يتم حساب العنوان باستخدام (مسجل +إزاحة) حيث لا مكان ثابت للعنوان في الذاكرة في هذا النوع من العنونة .

الإزاحة Offset :

هو موقع البيانات داخل أي مقطع من الذاكرة - بكل المقاطع موجودة الإزاحة - ، وهو البعد عن بداية المقطع ((وليس بداية الذاكرة))!!

جدول توضيح Offsetالمناسب لكل مقطع:

Segment	Offset (16-bit) 8080, 8086, 80286	Offset (32-bit) 80386 and above	Purpose
CS	IP	EIP	Program
SS	SP, BP	ESP, EBP	Stack
DS	BX, DI, SI, 8-bit or 16-bit #	EBX, EDI, ESI, EAX, ECX, EDX, 8-bit or 32-bit #	Data
ES	DI, with string instructions	EDI, with string instructions	String destination

CamScanner

وبما أنه لدينا أربع مقاطع نحتاج إلى أربع مسجلات لتحديد عناوين بداية المقاطع: segment register

> Code SegmentReg (CS)

> Data SegmantReg (DS)

> Extra SegmentReg (ES)

> Stack SegmentReg (SS)

> 16 BIT .. (FS & GS)

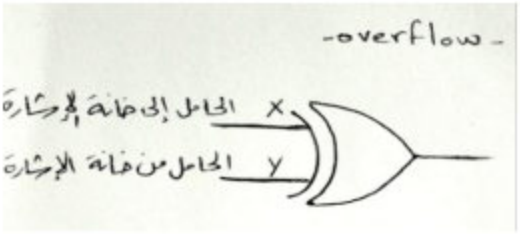


مسجلات الأعلام flags registers

هو مسجل أغراض خاصة كل بت فيه له وظيفة محددة ودارات محددة تحكم عليه و يمكن أيضاً التعامل معه ككتلة واحدة و يتأثر بالعمليات الحسابية والمنطقية (ويخزن نتائج العمليات و ليس الداتا) .

أشهرها :

- ✓ CF(Carry Flag): يخزن بداخله في حال وجود منقول في العمليات الحسابية .
- ✓ ZF (Zero Flag: اذا كان ناتج العملية صفر يخزن واحد .
- ✓ SF (Sign Flag) : يحدد اذا ما كانت الإشارة موجبة أو سالبة حيث : يخزن البت الأخير (أو قبل الأخير في حال وجود منقول).
- ✓ OF (Over Flow Flag): يخزن إذا حدث طفحان



- ✓ PF (Parity Flag): يستخدم في التحقق من زوجية وفردية الأعداد و هذا يساعد على اكتشاف الأخطاء في المعالجات ويتم هذا عن طريق بوابة XOR والخرج يكون Parity .
- ✓ AF (Auxiliary Carry) : يستخدم في عمليات محددة مثلاً يستخدم عند التعامل مع نظام BCD .

مثال تطبيقي :



بفرض نريد جمع عددين ...

(أعداد غير مؤشرة) :

$$\begin{array}{r} 129 \\ +\ 127 \\ \hline 256 \end{array}$$

(أعداد مؤشرة) :

$$\begin{array}{r} -127 \\ +127 \\ \hline 0 \end{array}$$

(بالنظام الثنائي) :

$$\begin{array}{r} 10000001 \\ +\ 01111111 \\ \hline 100000000 \end{array}$$

-في حالة الأعداد المؤشرة يكون : Z=1 <<< الناتج صفر .

C=1 <<< يوجد منقول في العملية .

S=0 <<< البت قبل الطفحان .

-في حالة الأعداد غير المؤشرة يكون :

Z=0 و C=1 و S=0 .

ملاحظة :

C=1 <<< خطأ عند التعامل مع أعداد غير مؤشرة

O=1 و C=1 <<< خطأ عند التعامل مع أعداد مؤشرة

ونلاحظ أن عدد المسجلات قليل مقارنة بمعالجات RISC التي قد يصل فيه عدد المسجلات فيها إلى 300 مسجل.

أعلام التحكم: (control flag)

نستطيع التحكم في سير البرنامج عن طريقها وهي :

1- D(direction):

هو flagمسؤول عن تحديد اتجاه العناوين حيث :

D=1 : باتجاه تناقص العناوين

D=0 : باتجاه تزايد العناوين

شرح: بفرض لدينا السلسلة النصية (Noon Team) ونريد عنونتها في الذاكرة

حيث 73 , 72 تمثل عناوين في الذاكرة



D=0
(بداية السلسة هنا من العنوان الأصغر)
عن طريق تعليمة CLD



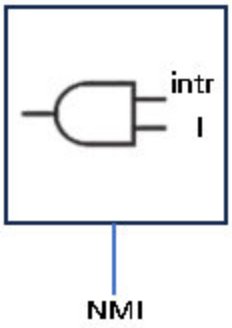
D=1
(بداية السلسة هنا من العنوان الأكبر)
عن طريق تعليمة STD

2- I (Interrupt):

لدينا نوعين من المقاطعة :

- Software : مثال عليها إرسال رسالة خطأ عند التقسيم على صفر (كود لا يكتبه المبرمج في لغات البرمجة عالية المستوى)
- Hardware : مثال عليها عند الضغط على زر في الكيبورد في أثناء عمل برنامج ما .

آلية العمل وحدة المعالجة المركزية (cpu) :



Intr: مدخل المقاطعة القابلة للحجب

المدخل I: اذا كان :

I=0 : يمنع تأهيل المقاطعة (سيتم حجب جميع المقاطعات)

I=1 : تأهيل المقاطعة

تقوم وحدة المعالجة المركزية CPUبفحص خرج بوابة AND لمعرفة اذا وجدت المقاطعة وهل يستجيب لها عن طريق إما تعليمة STI(set I)
أو CLI (clear I)

المدخل NMI (non mostable interrupt) المقاطعة غير لقابلة للحجب: يتم عبره إرسال إشارة إلى CPU لإيقاف ما تفعله .. أي تستخدم للتعامل مع الاحداث المهمة جدا .

3-Trap (single step debugging):

يوجد في المعالجات الحديثة :

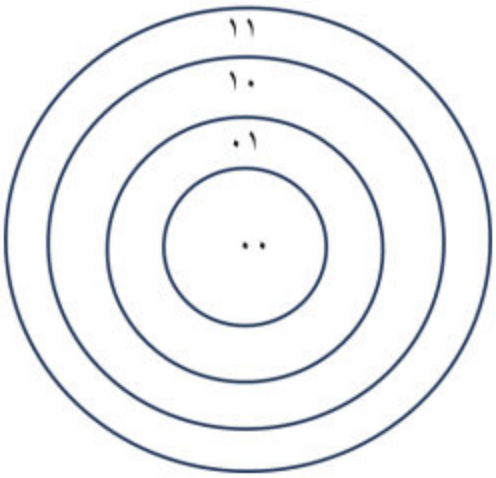
T=1: تأهيل single step flag(فحص البرنامج تعليمة تعليمة أي قيم السجلات , الذاكرة , حالة البرنامج)

T=0: منع تأهيل single step flag .

4-IOPL (privilege level in protected mode) :

مستويات الامتياز في نظام الحماية .

شرح: ما قبل اختراع نظام التشغيل windows وتحديدا مع معالج intel 8086 كان يستخدم نظام تشغيل Dos (شاشة سوداء دون أي رسوميات.. وينفذ برنامج واحد فقط) ومع تطور المعالجات واختراع نظام التشغيل windows واستخدام عدة برامج في وقت واحد أصبحنا نحتاج لزيادة الحماية لحقوق ملكية أكواد بعض البرامج الأساسية (مثل كود نظام التشغيل windows) وهذه هي وظيفة مستويات الامتياز وهي عبارة عن أربع حلقات حماية نمثلها ب 2bits وهي :



1- 00 (أعلى مستوى امتياز يصل لنظام التشغيل)
2- 01
3- 10
4- 11 (أخفض مستوى امتياز يقتصر على التطبيقات فقط).

ملاحظة :عندما يدخل البرنامج إلى الذاكرة يجب أن يكون مستوى الامتياز أعلى أو يساوي مستوى الذاكرة .ذ

5-(virtual mode) VM النمط الافتراضي :

عند تفعيله يعمل المعالج في النمط الافتراضي .

شرح :

لدينا ثلاث أنماط للمعالج:

النمط الحقيقي (Real mode):يتفعل عند إقلاع الحاسوب (شاشة Dos) يدعم نظام التشغيل 16 bit (معنى 16 bit: نظام تشغيل قادر على التعامل مع داتا وعناوين 16bit).



نمط الحماية (protected mode): يستخدم في أنظمة التشغيل الحديثة كنظام windows يدعم نظام تشغيل 32 bit وأكثر.

النمط الافتراضي (virtual mode) :وظيفته تشغيل عدة برامج Dos تحت نظام windows (كل منها محصور في قسم ذاكرة بحجم 1MB داخل نظام windows) .

وبهذا نكون انهينا الحديث عن النموذج البرمجي لمعالجات CISC وسننتقل إلى

العنونة Addressing

عنونة الذاكرة بالنمط الحقيقيReal mode memory addressing :

16bit (ابتداءً من intel 8086 وصولاً إلى intel 8088 " طول المسجلات ")

في معالجات 8086 يكون address bus هو 20bit ومن أجل ذلك تكون السعة الاعظمية للذاكرة (وهي الذاكرة الفعلية) $2^{20} = 1M$ وهي أكبر ذاكرة يمكن لهذا المعالج عنونتها .

- 16bit offset (بكل المعالجات .. ممكن تكون أقل " لهيك منكمل أصفار")

لدينا بعض القوانين للحصول على العناوين في الذاكرة :

1-العنوان الفعلي:

- $EA = Segment \times 10H + offset$

حيث:

segment: عنوان المقطع

offset: عنوان الازاحة

2- عنوان بداية المقطع starting address of segment:

$$s = segment \times 10H + 0000\ H$$

(بالسداسي عشر كل صفر هي أربع أصفار)

3-عنوان نهاية المقطع ending address of segment:

$$E = segment \times 10H + FFFF\ H$$

4-حجم المقطع:

$$Segment\ size = 2^{16(offset)} = 64KB$$

مثال تطبيقي:



بفرض لدينا

$$Segment = F23A$$
$$offset = 1000\ H$$

أحسب العنوان الفعلي:

الحل:

$$EA = F23A \times 10H + 1000H$$
$$= F33A0H(20bit)$$

سلبيات النمط الحقيقي:

عدم وجود حماية للذاكرة (لايوجد مستويات امتياز)

سعة ذاكرة صغيرة (1MB) فقط

تأخير زمني ناتج عن الضرب ب 10H (عن طريقة الازاحة لليسار)

يجب أن يبدأ العنوان بصفر حصراً (تم حل المشكلة في نمط الحماية

طرق العنونة Addressing mode

سؤال: ماهو المقصود بطرق العنونة؟!

هي كيفية الوصول لموقع في الذاكرة وكل ما كثرت هذه الأنماط جعلت كتابة التعليمات البرمجية أسهل على المبرمج وزادت الصعوبة في التصنيع

الطريقة الأولى: العنونة بالسجل (*Register*):

توضيح: (تعليمة MOV هي ابسط تعليمة بلغة التجميع (لغة برمجة منخفضة المستوى) سنشرح عليها كافة طرق العنونة على سبيل المثال)

$$MOV\ AX, BX$$

معنى التعليمة: يتم أخذ المحتوى الموجود في السجل BX وتخزينه في السجل AX (**ويجب أن تكون أطوال المسجلات مثل بعضها**)

ملاحظة هامة جداً:



من الخطأ أن نكتب: $MOV\ IP, AX$
والسبب أن هذا المسجل يحوي على عنوان التعليمة التالية التي سوف تتنفذ وأي تغيير في تسلسل هذه القيم سيؤدي إلى أخطاء في البرنامج ولكن من الممكن أن نكتب $MOV\ AX, IP$ أي ممكن تخزين عنوان تعليمة في مسجل AX دون مشاكل

الطريقة الثانية: *Immediate*:

يخزن قيمة داخل مسجل

$$MOV\ CH, 34H$$

انتبه!! لا يمكن أن تكون القيمة المخزنة طولها أكثر من 8 بت ((لأن المسجل CH طوله 8بت وما بيوسع اكتر)) ولكن من الممكن أن يكون المسجل أكبر من 8 بت ونسجل قيم طولها أقل ((بالزيادة نضع أصفار))

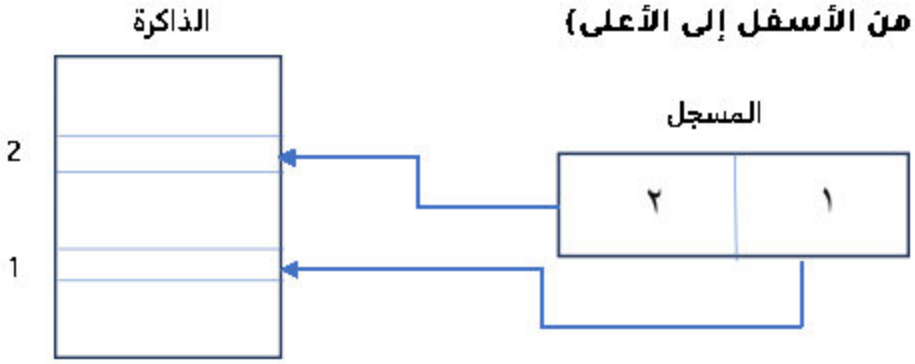
الطريقة الثالثة: العنونة المباشرة للذاكرة *direct*:

وهي نقل من مسجل إلى ذاكرة

$$MOV\ [234H], AX$$

ملاحظة: قوسين[] = عنوان

وتوضع الداتا في العنوان الأدنى في البايت الأدنى والداتا في العنوان الأعلى في البايت الأعلى (العد من الأسفل إلى الأعلى)



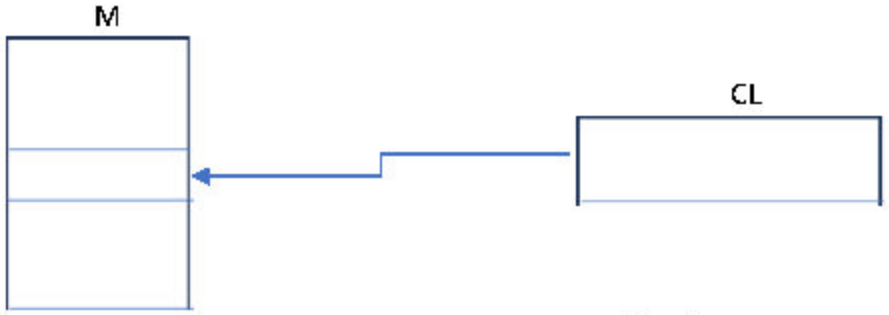
ملاحظة: لا نحصل على العناوين الفيزيائية إلا بعد وضع التعليمة في الذاكرة

$$DS \times 10H + 234H$$

الطريقة الرابعة: العنونة غير المباشرة بالمسجل *Register Indirect*:

$$MOV\ [BX], CL$$





نقل من المسجل إلى الذاكرة

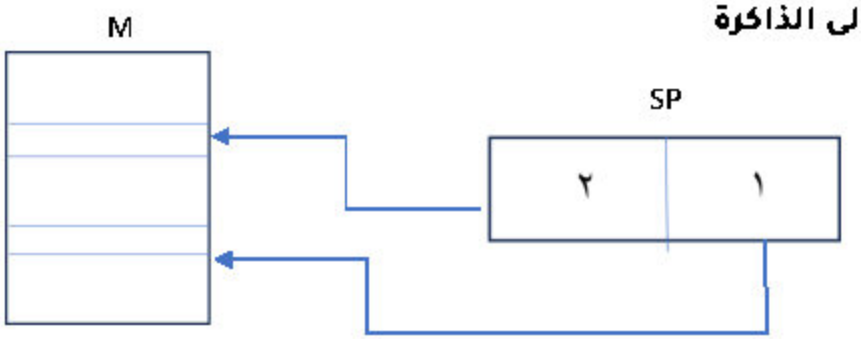
لحساب عنوان BX:

$$DS \times 10H + BX$$

الطريقة الخامسة:Base plus Index

$$MOV\ [BX + SI], SP$$

من المسجل إلى الذاكرة



لايجاد العنوان

$$DS \times 10H + BX + SI$$

والعنوان الذي يليه

$$DS \times 10H + BX + SI + 1$$

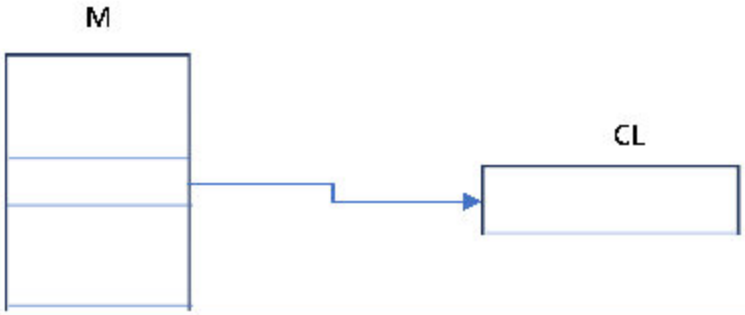
الطريقة السادسة: Register relative:

$$MOV\ CL, [BX + 4]$$

إزاحة أربع مرات بالنسبة للمسجل

لايجاد العنوان:

$$DS \times 10H + BX + 4$$



الطريقة السابعة: Base relative plus Index:

من مسجل إلى ذاكرة

$$MOV\ ARRAY[BX + SI], DX$$

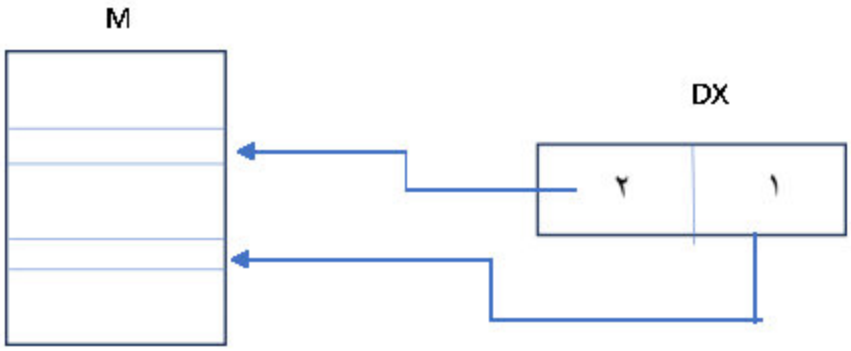
(تعرف المصفوفة من قبل المبرمج)

لإيجاد العنوان

$$DS \times 10H + ARRAY + BX + SI$$

والعنوان التالي:

$$DS \times 10H + ARRAY + BX + SI + 1$$



الطريقة الثامنة: *Scaled Index*:

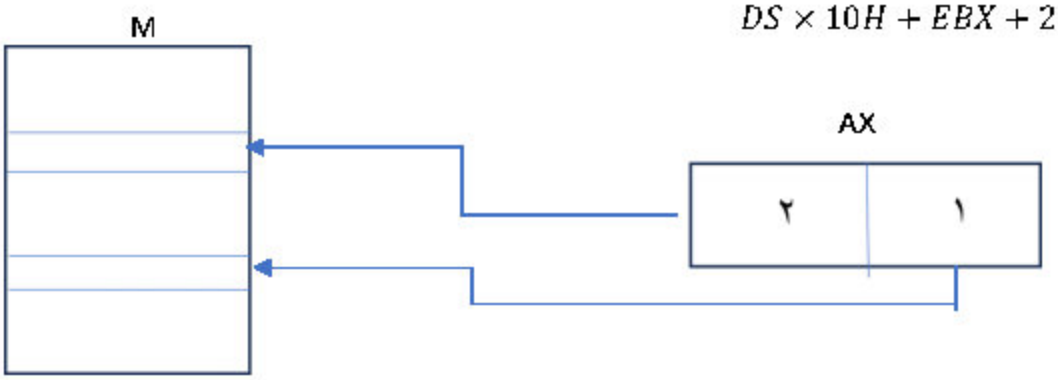
$$MOV\ [EBX + 2 \times ESI], AX$$

لإيجاد العنوان:

$$DS \times 10H + EBX + 2 \times ESI$$

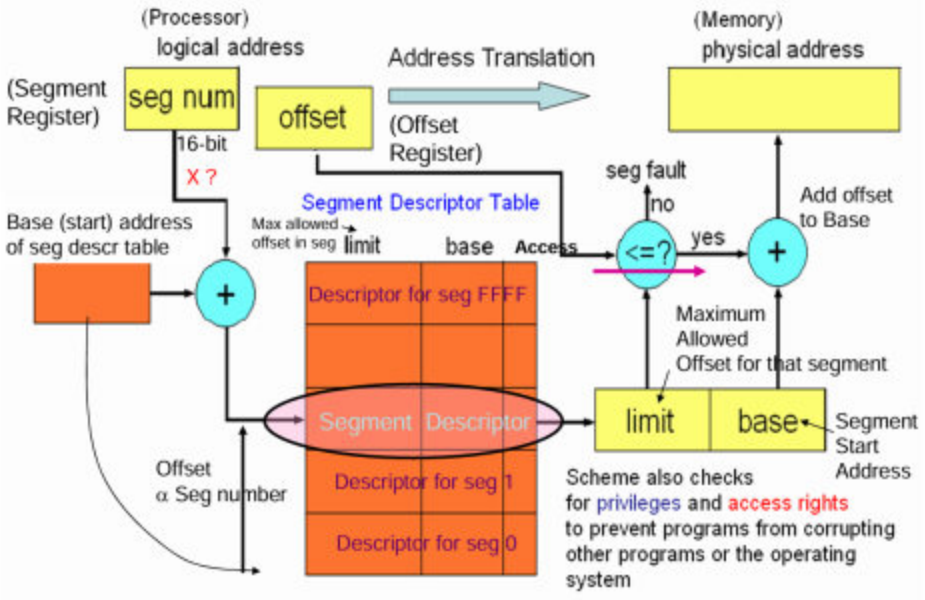
العنوان التالي:

$$DS \times 10H + EBX + 2 \times ESI + 1$$



نمط الحماية (Protected mode)

بدء اعتباراً من معالج intel 8086 بشكل خفيف وظلّ مستمراً وصولاً الى معالجات core .



تقسيم المقاطع في الذاكرة في هذا الوضع :

أولاً: المقاطع segments:

كل مقطع 16 bit تقسم على الشكل التالي :

13	1	2

16 bit

2 bit هي RPL (Requested Privilege Level) أي طلب دخول لمستويات الامتياز

1 bit هو TI (Table Index) أي دليل الجدول ويكون هذا البت:

- إما 0 : ندخل إلى جدول اسمه GDT (Global discripter table) وهو جدول واحد لكل البرامج " المعلومات التي فيه تهم كل البرامج " .
- أو 1 : ندخل الى جدول اسمه (LDT (Local discripter table حيث لكل برنامج جدول واحد

مثال توضيحي :
بفرض أننا قمنا بتشغيل ثلاث برامج سيكون لدينا ثلاث جداول من نوع LDT لكل برنامج ؛ وجدول واحد GDT لكل البرامج .

13 bit هو selector: عندما نريد الدخول الى مقطع في الذاكرة يتم ذلك عن طريق جدول DT وإما يكون GDT أو LDT ويكون عدد Discripters $= 8192 = 2^{13}$ ومنه الـ 16 bit .

يحوي توصيف ما يلي :

- عنوان بداية المقطع (Base) .
- limite(Max of offset) الحد الأعظم للإزاحة .



ملاحظة : لا نضع عنوان نهاية المقطع لتصغير عدد البتات المطلوبة .

مثال توضيحي : بفرض لدينا 11 موقع في الذاكرة والذاكرة طولها 11 byte يكون $limit = 10$

أي نبدأ العد من الصفر ؛ ويكون قياس المقطع بدلالة limit

$$size\ of\ seg = limit + 1$$

$$end = Base + limit$$

- حقوق الوصول (AR(Access rights .

يوجد للمقطع مستوى امتياز خاص به ؛ ومستوى امتياز للواصف
يتم مقارنة مستوى الامتياز لكل منهما ..

إذا كان مستوى الامتياز للواصف يساوي أو أقوى من مستوى المقطع . يتم الدخول الى المقطع .ولا يمكن الدخول للمقطع بخلاف ذلك .

بعد التأكد من مستوى الامتياز والدخول الى مقطع الذاكرة ؛نتذكر أن المقطع يحوي Data (مقاطع , data ,stack,exta) ويحوي code



يكون من صلاحيات الوصول للداتا :

1- حقوق Read,Write (قراءة وتغيير الداتا)

2- قراءة فقط Read only (ممنوع التغيير في الداتا)

من صلاحيات الوصول للكود :

1- تنفيذ الكود (execute).

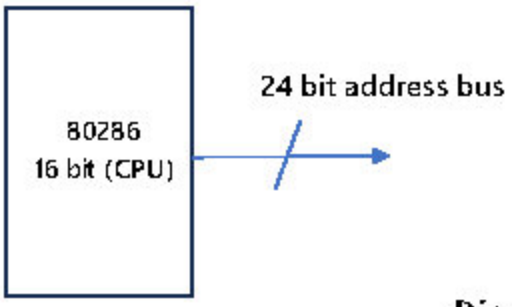
2- تعديل الكود .

3- تنفيذ الكود دون قراءته (execute only)

ملاحظة: في نمط الحماية لم نضرب بـ 10 ؛ وأصبحت في الذاكرة بشكل مباشر بل مررنا بمستويات امتياز و حقوق الوصول .

ثانياً: الإزاحة (Offset) :

في معالجات intel 80286 كان طولها 16 bit (CPU)



شرح :

أول معالج 16 bit يدعم نمط الحماية

طول $offset = 16\,bit$

ويكون فيه طول لوصف

$$Discripter\ size = 8\ byte$$

فعلياً توجد الذاكرة بالشكل التالي :

0
0
AR
Base
Base
Base
limit
limit

ولكن لسهولة الشرح سنرسم على 2 byte بجانب بعضهما :

00000000	00000000
AR	Base B23....B1
Base B15....B8	Base B7....B0
Limit	Limit

في أول 2 byte نضع limit ؛ لان limit هو نفسه offset طوله 16 bit

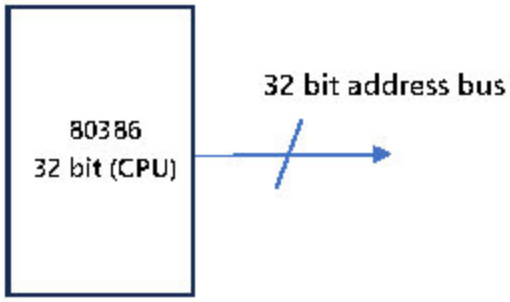
- Base: هي العنوان الفيزيائي طولها 24bitأي تحتاج لحجز 3 byte
- AR: يحجز بايت واحد .. وسنشرح توزيع البتات داخله لاحقاً .

البايتين الاضافيين أضيفت من قبل مهندسي intel احتياطا لتطوير قادم .

التوصيف في معالج Intel 80386:

معالجات 80386 أصبحت 32 bit (CPU).

أول معالج 32 bit يدعم نمط الحماية وصولاً الى p4 ومن أجل تشغيل البرامج القديمة على هذا المعالج نحافظ على شكل التوصيل بمعالج 80286. ونطور عليه حيث يصبح لحل مشكلة العنوان نأخذ 1 byte ليصبح 32 bit



ولحل مشكلة 32 bit =offset نحتاج الى 2byte ولم يبقى لدينا إلا 1byte فارغ وهنا ظهرت مشكلة؛ نقوم بحلها عن طريق قسم الـ byte الى قسمين متساويين 4 bit هي limit لنحصل على offset = 20 bit وتم تقسيم الـ 4bit الأخرى كل واحد بت .

Base	G	D	0	AV	Limit
AR	Base				
Base	Base				
Limit	Limit				

- البت الأول : AV (Avaliability segment) توافرية الذاكرة .
- البت الثاني : يخزن قيمة صفر ؛ ولا يتغير أبدا (ممنوع لمبرمج النظام الدخول اليها)
- البت الثالث : Dinstructia mode نمط العنونة .
- البت الرابع : Granularity مقدار الخشونة

شرح :

يستفاد منه مبرمج النظام من البت الأول إذا كان AV =1 فإن المقطع يكون حاجز للاستخدام وإذا كان AV =0 يصبح هذا المقطع غير متاح

البت الثالث في معالج 80386 يجب أن يكون طول الداتا وطول الازاحة 32 bit لأن طول المسجلات داخل cpu =32 bit

- ومنه عندما D =1 نأخذ طول الداتا وطول الازاحة 32 bit (وهو نمط الطبيعي)
- وعندما D =0 طول الداتا وطول الازاحة 16 bit (من اجل تشغيل البرامج القديمة)

البت الرابع : من أجل حل مشكلة نقص البتات في limit يفحص هذا البت

G =0 اذا اردنا استخدام مقطع من 1byte إلى 1MB وتكون $limit = 20bit$ أي في الوضع الطبيعي

G=1 يصبح لدينا:

$Limit =$ اول قيمة $00000FFF$ ... أخر قيمة $FFFFFFFF$

بهذه الطريقة يصبح حجم المقطع من 1MB الى 4GB



مثال تطبيقي : بفرض لدينا التعليمة

Mov AX , BX

توصف التعليمة داخل المعالج بالشكل التالي :

يكون رمز AX = 000 , EAX = 000 المعالج يرى أن EAX , AX متشابهان وهنا نلجأالى بت D

اذا كان D = 1 : سيعتبرها EAX (الاحدث)

D = 0 : سيعتبرها AX (الاقدم)



تمرين 1:

ليكن لدينا معالج Intel-286 ولدينا Ds = 0008H المطلوب احسب :

Base,End

هل جدول التوصيف GDT ام LDT ؟

الحل :

نفكك Ds

0000 0000 0000 1000

00: اعلى مستوى امتياز

GTD:0

00001 0000 0000: نذهب إلى Discripterرقم 1

للحصول على عنوان اول بايت في ال Discripter نضرب بـ8 عن طريق الازاحة لليسار ثلاث خانات , وللدلالة على ذلك نكتب الرمز 0006 .

60
00
92
10
00
00
00
ff

لحساب limit

هو اول بايتين limit = 00ff

$$Base = 100000$$

$$Size\ of\ GDT = 2^{13} \times 2^{3} = 64kB$$

$$Size\ of\ LDT = 2^{13} \times 2^{3} = 64kB$$



تمرين 2 :

لدينا Discvipter

intel 80386
- base= 23 00 00 00 H (32 bit)
- Limit = 01 2F FH (20 bit)

قد يطلب معرفة نوع المعالج

أوجد عنوان بداية المقطع وعنوان نهاية المقطع وحجم المقطع.

الحل :

$$If\ G = 0$$

عنوان بداية المقطع = $2300\ 00\ 00H$

عنوان نهاية المقطع = $base + limit$

$=2300\ 00\ 00H + 012FFH = 230012FFH$

حجم المقطع $1 + Limit =$

$= 12FFH + 1H = 1300H$

$$G = 1 \text{ ومن أجل}$$

يكون $Limit = 012FF\ FFF$

عنوان بداية المقطع = $= 23\ 0000\ 0H$

$$\text{عنوان نهاية المقطع} = 012FF\ FFFH + 2300\ 0000H = 242FFFFFH$$

حجم المقطع = $12FFFFF + 1H = 1300\ 000H = 2^{12} \times 1300H = 4K * 1300H$

حقوق الوصول AR(Access Right)

تحدثنا في المحاضرة السابقة عن نمط الحماية واستعراض تقسيم الواصف في معالجات مختلفة والتي كانت تحوي 1 Byte والتي تشمل حقوق الوصول AR. في هذه المحاضرة سنتوسع في شرح هذا البايت.



- حيث يأخذ بايت AR التوزع التالي :

P	DPL	S	E,ED/C,W/R	A
1 bit	2 bit	1 bit	3bit	1 bit

8Bit

P (present)

عند طلب الدخول الى مقطع في الذاكرة حيث اذا كان :

- P=0 يدل على ان المقطع المطلوب غير موجود في الذاكرة.
- P=1 يدل على ان المقطع المطلوب موجود في الذاكرة يتمك إعطاء و جلب البرنامج من الهارد الى الواصف.

DPL (Descriptor Privilege Level)

تتم المقارنة بين هذين البتين مع RPL (طلب الدخول لمستويات الامتياز).

في حالة كانت متساوية او اقوى سيتم الدخول الى مقطع الذاكرة.

ايضاً نميز حالتين:

- S=1 يدل على ان المقطع يحتوي اما data, stack, code
- S=0 تدل على system descriptor

شرح :

لدينا البرنامجين التاليين

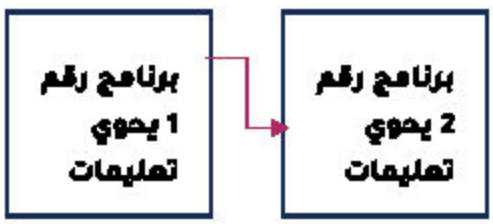
يتيح لنا نظام التشغيل الانتقال من برنامج الى آخر.

بفرض اننا نستخدم البرنامج الأول ونريد الانتقال الى البرنامج الثاني يتعين علينا تخزين تعليمات البرنامج الاول لاستخدامها لاحقا

يتم تخزين تعليمات البرنامج الاول في مقاطع خاصة

تخزن حالة البرنامج TSS

تتم العملية عن طريق مسجل TR طوله 16 بت يأخذ توزع البتات التالي



حيث اخر 13 بت تمثل عنوان الواصف

13 bit	1 bit	2 bit

من اجل حفظ حالة البرنامج الأول وبما اننا نتعامل مع نمط الحماية يتم اخذ عنوان الواصف GDTمن TR لننتقل الى الواصف ومنه سيتم تخزين التعليمات فيTSS

من اجل تحديد عنوان بداية GDT يتم هذا عن طريق مسجل GDTRو يحدد limit حجم هذا الجدول.

(base)	(limit)
gi32bit 24bit	16 bit

ملحوظة : ان عدد جداول ال LDT غير محدد, السبب هو اننا نحتاج الى جدول لكل برنامج (أي سيختلف العدد بعدد البرامج المفتوحة).

يتم حل المشكلة عن طريق ما يعرف بال Local Descriptor Table Segment (LDTS)

- من جدول ال LDT يوجد مقطع يحتوى على معلومات عن موقع البرنامج و لدينا مسجل LDTR وحيد لكل برنامج.

13	1	2

يدل على بداية LDT

LDTR

- E,ED/C,W/R

ال 3 بت تمثل نوع المقطع, لها 8 احتمالات تبدأ من 000 نهايةً ب 111 و سنشرح دلالة كل منها بالتفصيل

الخوارزمية المعتمدة:

اول بت من اليسار: يحدد ان كنا سنتعامل مع data او code

ثاني بت: في حالة التعامل مع Data

Data :0

Stack:1



في حال كان Code

(1: قارن مستوي الامتياز

0: اهمل مستوي الامتياز)

البت الثالث : (0: قراءة دون تعديل, 1: قراءة مع تعديل)

1. Data reading only 000
2. Data (could be reading or writing) 001
3. Stack reading only 010
4. Stack (could be reading or writing) 011
5. Code execute only 100
6. Code execute reading 101
7. Code execute only 110
8. Code execute reading 111



مثال تطبيقي شامل (هام جداً) :

بفرض لدينا AR = (92)H والمطلوب:

1.حدد نوع المقطع.

2.هل هي موجودة في الذاكرة ام لا.

3.بفرض ان RPL=10 هل سنتمكن من الدخول الى الذاكرة؟ حدد نوع المقطع.(خارجي)

4.هل المقطع يحتوي على بيانات او اكواد او يخزن حالة برنامج معين؟ (خارجي)

5.هل تتم العنونة في المقطع باتجاه زيادة العناوين او تناقص العناوين؟ (خارجي)

الحل:

(10010010)B = (92)H

1. نوع المقطع:

نلاحظ ان 10010010 ان المقطع من النوع داتا قابلة للقراءة و التعديل

2. نلاحظ من 10010010 ان المقطع موجود في الذاكرة

3. نلاحظ من 10010010 و بالمقارنة بين 00 و 10 حيث نعلم ان 00 اقرب من 10 و بالتالي نستطيع الوصول الى الذاكرة.

4. من 10010010 نلاحظ انه يصف بيانات و اكواد.

5. بما ان التعامل مع DATA وليس STACK تتم العنونة باتجاه تزايد العناوين.



مفتاح الحل:

1 نحول من السداسي عشر إلى النظام الثنائي

2 نفحص البتات حسب التوزع التالي(حفظ):

A	E,ED/C,W/R	S	DPL	P
1 bit	3bit	1 bit	2 bit	1 bit

3 نجاوب على المطلوب بحسب الشرح السابق

A(Access)

و نميز ايضاً حالتين:

- A=0 تم الوصول للذاكرة.
- A=1 لم يصل المقطع الى الذاكرة.

Interrupt descriptor table

عرفت شركة Intel حوالي 256 نوعاً من المقاطعة منقسمة الى مقاطعات software و hardware و بما اننا ما زلنا نتحدث عن نمط الحماية سنحتاج الى واصف لهذه المقاطعات للدخول الى الذاكرة.

يتم تحديد عنوان بداية جدول IDT عن طريق مسجل IDTR (مسجل فيزيائي).

وبهذا نكون انهينا الحديث عن نمط الحماية وسننتقل إلى

تقسيم الصفحات Paging

تستخدم للتقسيم الصفحي. حيث بدأت من معالج intel 386(Adress bus 32)





شرح بسيط لمفهوم paging : نتعامل في الذاكرة مع مقاطع في البرامج المختلفة يكون طول كل برنامج مختلف عن الاخرلذلك عندما نريد احضار المعلومات من الهارد الى مقطع ما في الذاكرة تكون اطوال البيانات مختلفة في كل مقطع. و لاجل ذلك يتم احضار اقسام متساوية من البيانات غالبا 4KB في كل مرة تنتقل من الهارد الى الذاكرة .

أيضا: عند امتلاء الذاكرة يقوم نظام التشغيل بحجز مساحة من القرص الصلب ويعاملها كأنها امتداد لهذه الذاكرة وهنا يظهر مفهوم الذاكرة الافتراضية حيث نعتبر المساحة التي يشغلها القرص الصلب هي ذاكرة وهمية أو افتراضية

لمعرفة عدد الـ *Fages* في الذاكرة الافتراضية :

$$\frac{4GB}{4KB} = 1M\text{صفحة}$$

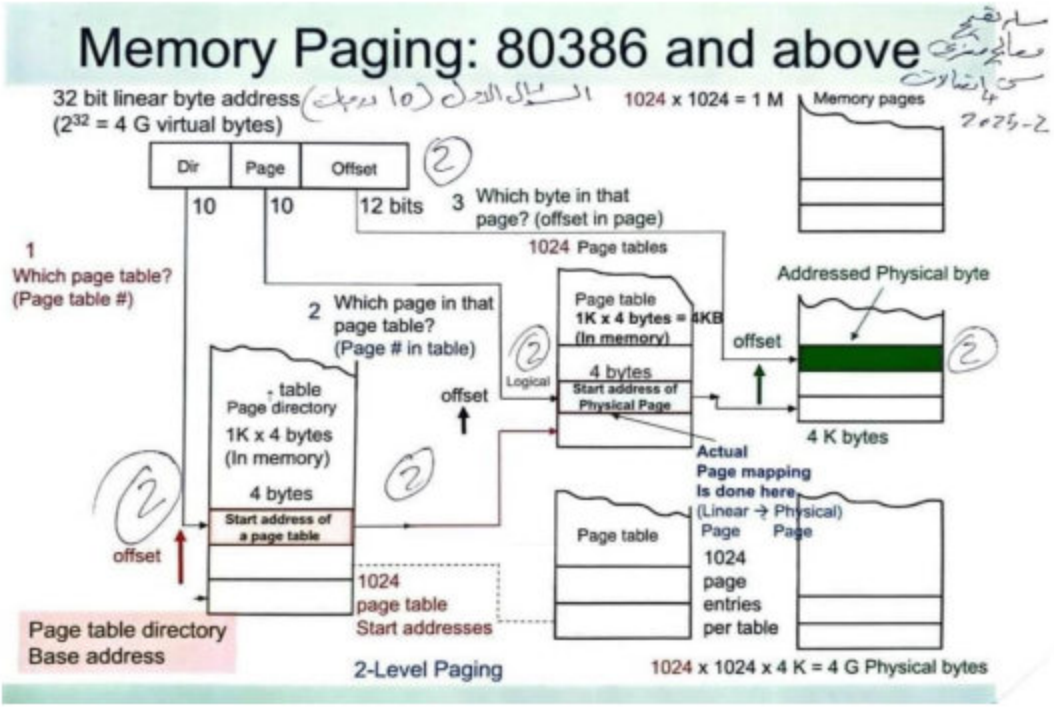
حجم الهارد الكلي

سعة الصفحة الواحدة

ملاحظة:

في معالجات intel يمكن التحكم بسعة تخزين الصفحة الواحدة لتصبح إما 4KB أو 4MB
تسمى الصفحة الموجودة في الذاكرة Physical page(frame)
تسمى الصفحة الموجودة على القرص الصلب logical page وهي صفحة افتراضية

شرح:



ذكرنا سابقاً أننا عندما نتعامل مع نمط الحماية لانتقال أي معلومات إلى الذاكرة يتم جمع اللإزاحة offset مع base في DT لتحصل على linear address(logical) (العنوان الخطي)

في حال لم يتم تقسيم الصفحات في الهارد يكون:

Linear address =physical address(العنوان الفعلي في الذاكرة)

في حال تم قسم الصفحات في الهارد يكون

Linear address ≠physical address(العنوان الفعلي في الذاكرة)

حيث يتم التحكم في إمكانية تقسيم الصفحات من خلال مسجل CR0(Control Register zero)

اذا كان:

PG=0 : لا يتم تفعيل تقسيم الصفحات no paging

PG=1: يتم تفعيل تقسيم الصفحات Paging

PG 1bit	

لدينا نوعين من تقسيم الصفحات:

One level paging:

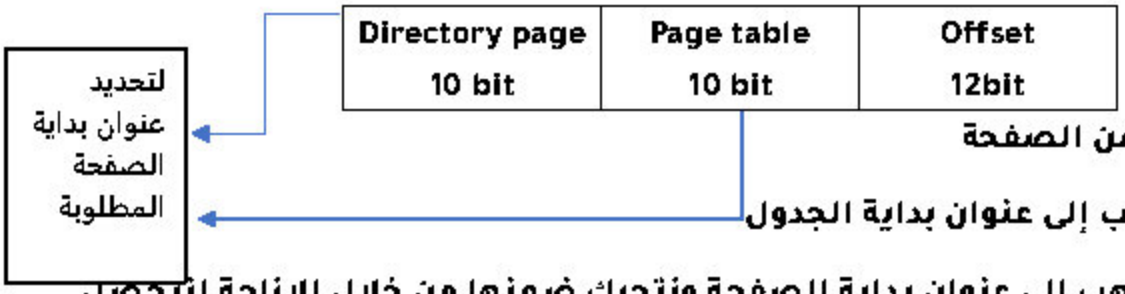
يعتمد على وضع 1M عنوان بداية صفحة في جدول واحد ولكن بهذه الطريقة ستطول عملية البحث عند فتح عدة برامج

من اجل هذا تم حل المشكلة في النوع الثاني وهو

Tow level paging:

وهو المستخدم في شركة intel يعتمد على مبدأ تقسيم الصفحات على عدة جداول كمثال:

في حالة تقسيم الصفحات يكون طول العنوانالخطي يأخذ التوزع التالي:



Offset: للتحرك ضمن الصفحة

المرحلة الأولى:نذهب إلى عنوان بداية الجدول

المرحلة الثانية : نذهب إلى عنوان بداية الصفحة ونتحرك ضمنها من خلال الإزاحة لنحصل على العنوان المطلوب

ملاحظة: في كلا المرحلتين نضرب ب 4 والسبب أنه يوجد 4Byte بين كل عنوان وعنوان



مثال بسيط لتوضيح الفكرة:
نعتبر الجدول أحادي المستوى ك كتاب ضخم يحتوي على مليون صفحة، إذا أردت البحث فيه ستضيع. بدلاً من ذلك، قمنا بتحويله إلى مكتبة (جدول متعدد المستويات):

نستخدم مسجل CR3 من أجل تحديد عنوان بداية Directory page وهو المسؤول عن ترجمة العناوين الوهمية في الصفحات (في الهارد) لتصبح عناوين حقيقية (في الذاكرة) يأخذ هذا المسجل التوزع التالي:

10 bit	10 bit	12bit

ملاحظة هامة جداً: ممنوع ان نستخدم أول 12 بت لأنها تعطي معلومات كاملة عن الصفحة أي لا يتم استخدامها كعنوان

مثال تطبيقي هام:



بفرض لدينا في المسجل CR3

$$CR3 = (C1001A2F)_H$$

أوجد عنوان بداية Directory table؟

الحل:

نقوم بتصفير اول 12 بت على اليمين فيصبح لدينا:

$$CR3 = (C1001000)_H$$

انتهى الملحق الخامس ...

اكتب ملاحظاتك (●'◡'●):

فريقنا يتمنى لكم دراسة موفقة